Self-Reconciling Data Pipeline — Short Runbook

Oct 27, 2026 · @Bhavna

A data pipeline on Google Cloud that checks every load against the source's own receipt, fixes what is safe to fix, holds what is not, and asks a person when a decision changes meaning — proven on synthetic hospital data. This is the short version for business and technical readers; the full build runbook has every command and screenshot.

1. In one page

Bad data is never published, and failures we understand fix themselves and are checked again. We built that around a dbt and Airflow pipeline on Google Cloud. We proved it live on synthetic hospital data: all 23 failure types in the design's decision table, each through the real pipeline, with no wrong figure reaching a report.

The problem. Today a pipeline proves that it *ran*, not that the data is *right*. A nightly load can finish green while 800 hospital visits are missing. Often the first person to notice is a ward manager reading a wrong bed report. Recovery is manual. L1 support searches logs for about 30 minutes, then hands over to L2 (the data owner) or L3 (engineers). When a source system goes down, every dataset fails on its own and on-call gets twenty alerts for one problem.

What we built. A control loop around the pipeline: **detect → hold → decide → fix → verify**.

Every load is checked against a receipt taken from the source. Data is held in `audit` until it passes. A deterministic engine picks one safe action, asks the data owner in Slack, or hands over to engineers, and closes an incident only after a re-check. AI agents only advise. A source outage becomes one incident, not twenty. Section 2 shows how the parts fit.

The promise to report users: you may see slightly old figures, but never wrong ones, and late data is always marked as late.

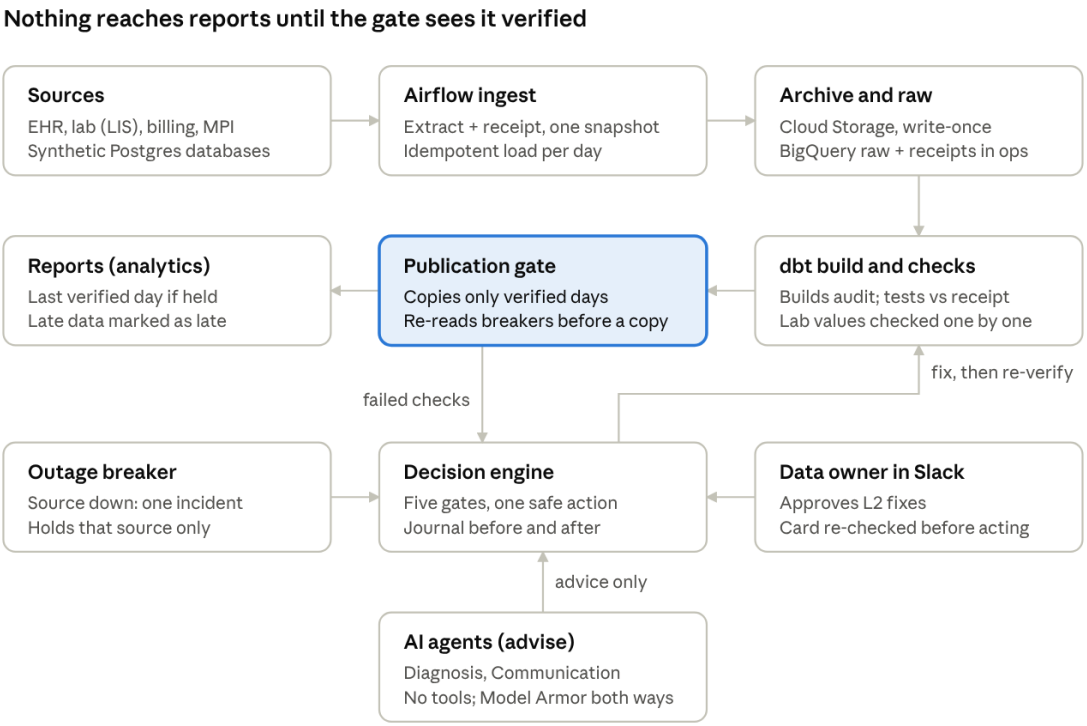
Results on 27 Oct 2026

Measure	Result
Failure types proven end to end	23 of 23
Wrong figures published during the failure runs	0
Incidents in 7 days / handled with no person	77 / 19
Median time to resolve	3.9 min
Patient data in agent calls, cards or the journal (T26)	0 findings in 18 checks
Hardening checks	17 pass, 1 warning, 0 fail

Who this is for. Business readers need sections 1, 3, 7 and 11. Data engineers and solution architects should read all of it. AI engineers should start with sections 5 and 6.

2. How it works

Data flows left to right along the top and comes back through the gate. The control loop underneath decides what happens when a check fails.



architecture · data path above, control loop below

Each night Airflow extracts every source table together with its receipt from the same snapshot. The extract is stored write-once in Cloud Storage and loaded into BigQuery `raw`. dbt builds `audit` and tests it against the receipts. The gate copies only verified days to `analytics`. A failed check goes to the decision engine. The engine fixes the problem and sends the day back through dbt and the gate, or asks the data owner, or hands over to engineers. Operators watch it all in the console, which only reads.

Built with Airflow 3.3.2 in Docker, dbt on BigQuery (run from Airflow tasks), Cloud Storage, Cloud Run (console, ADK agent service, Slack callback), Cloud SQL, Cloud KMS, Secret Manager, Model Armor, Gemini 3.5 on Vertex AI in asia-south1, Terraform and GitHub Actions CI. BigQuery holds `raw`, `audit`, `analytics`, `quarantine`, `ops`, `evidence_restricted`, `publication` and `recovery` (the incident journal).

3. How a decision is made

The engine is deterministic: the same evidence always gives the same decision. So every decision can be replayed, audited and explained. It asks five questions in a fixed order, and every answer is written to the incident journal with the policy version and the evidence.



decision engine · five gates, five exits

The policy (`policy/policy.yml`) runs in three stages, so no ordinary rule can override safety. Safety overrides come first, such as an open outage breaker. Ordinary rules come next, and the first match wins. The publication gate comes last. Changing a rule takes a reviewed pull request and a new policy version, and CI validates the file.

Severity sets the urgency. P1 means critical published data is wrong or missing: the safe fix runs and the data owner is paged at once. P2 means important data is delayed, with the last verified day still showing: the engine has 30 minutes to fix it before escalating. P3 means low impact, caught before publication.

Severity rises when data stays stale past its freshness deadline, so nothing waits unseen.

4. The failures it handles

Every failure gets one of five responses, and only the first one changes data without a person. The table groups the failure types we proved live by response. Each one has a rule in `policy/policy.yml` (version 1.8.0-poc.5) and a suite case that injects it.

Response	Failure types and what happens
Fixes itself (L1, re-verified)	**partial_extract**, **offsetting_errors**: rows missing or duplicated against the receipt → re-extract. **duplicate_batch**, **late_change**,

Response	Failure types and what happens
	change_beyond_lookback: a day loaded twice or changed later at source → rebuild that day. **young_orphan**: a lab result before its visit → link to the unknown member. **source_timeout** → Airflow retries
Waits (no data change)	**stale_feed**: replica hours behind. **late_documentation**: only part of the day in the source yet → hold and re-check
Expected, logged	**archived_history**, **archived_parent**: the EHR archived old visits → log as expected and publish
A person approves (L2, Slack)	**merged_patient_orphan**: records on a retired patient ID → remap after approval. **control_totals_missing**: data with no receipt → re-extract after approval
Held and handed over (L3 unless noted)	**unit_change** (glucose in mmol/L), **value_mismatch** (lab values rounded), **suspected_regression** (a release doubled rows; release attached), **build_error**, **schema_drift**, **staging_duplicates**, **join_fanout**, **checks_did_not_run**; **true_orphan** goes to the data owner (L2). A report depending on a held dataset stays on its last verified day
Grouped	**source_outage**: one source fails across datasets → one incident, that source held, others publish. **global_outage**: two or more sources down → every report held
Unknown	**fallback_unknown** → the AI agent suggests a cause; the engine validates it, otherwise L2

Never automatic, whatever any rule or agent says: `full_refresh`, `rollback` and `source_delete`.

5. Safety by design

Safety comes from permissions and fixed rules, not from good intentions in the code. Even with a code bug, the platform cannot publish unverified data, because only the publish identity may write `analytics`.

Guardrail	How it is enforced (proof)
One identity per component	Each part runs as its own service account built by Terraform; sa-dbt was denied writing `analytics` (T9)

Guardrail	How it is enforced (proof)
No key files	Operators impersonate service accounts for an hour; no user-managed keys exist
Write-Audit-Publish	Only verified days are copied; the gate re-reads breakers before every copy (T8, T20)
Fixed action list	Only catalog actions run; `full_refresh`, `rollback`, `source_delete` are never automatic (CI checks the policy)
Attempt cap and kill switch	At most 2 automatic attempts per incident; one setting turns every fix into a recommendation
Approvals re-checked	Fingerprint, incident state, policy version, kill switch, cap and evidence are re-checked; stale or expired cards are refused (T24)
Idempotent loads, immutable archive	One transaction per day load (T1); every extract kept write-once for replay
No patient data to AI	Masked views; evidence holds keys and field names, never values (T26: 0 findings in 18 checks)
Model Armor	Screens every agent request and answer in asia-south1; fails closed (T16)
Private by default	Everything in asia-south1; only the Slack callback is public, and Slack signs each request; secrets only in Secret Manager

Data stays synthetic. `agents.enabled` and `residency_confirmed` stay false for real data, and the hardening pass fails if either is true. Real data needs written confirmation of India residency, BAA cover and no training on prompts.

6. The AI agents

The agents advise and the engine decides. An agent is asked only when no rule explains a failure (`fallback_unknown`). Its answer goes through the same five gates as a rule. Today the agents run on synthetic data and only recommend (owner decision POC-SYNTHETIC-ONLY).

Agent	Job and where it runs
Diagnosis (gemini-3.5-flash)	Explains an unknown failure and proposes one catalog action or `escalate`, with confidence and cited evidence. ADK app `reconcile_agents` on the private Cloud Run service `reconcile-diagnosis`
Communication (gemini-3.5-flash-lite)	Words the summary on an approval card; called from

Agent	Job and where it runs
	the pipeline's agent gateway
Assistant (gemini-3.5-flash)	"Ask the pipeline" on the Operations screen; ADK app `reconcile_assistant` on the same service; off until switched on

The registry (`agents/registry.yml`) pins every model, prompt version, budget and timeout. CI fails on a retired model or a region outside Mumbai.

How an agent call is made safe

1. The gateway builds a masked summary: counts and check names only, no patient data.
2. Model Armor screens the untrusted log and check text. On injection or sensitive data the model is never called.
3. The agent answers in strict JSON. It has **no tools**, so it cannot read or change anything itself.
4. Model Armor screens the answer. The engine then validates it: JSON shape, action in the catalog and allowed for this agent, not never-automatic, evidence cited, scope, confidence of at least 0.80, and a 20-second limit.
5. Anything rejected or timed out is escalated to a person, and the run is not delayed (T15–T17).
6. Every call is a row in `recovery.agent_calls`: agent, model, prompt and policy version, result and reason, latency. For the assistant, only the question's length is stored, never its text; the answer is kept for audit and is covered by the T26 scan.

The assistant is read-only by construction. The console API gathers the context itself: report states, incidents, cards, breakers and policy, as counts and engine notes. It sends that with the question. Every answer must list the incident or card IDs it used. An answer that names an ID not in the context is rejected.

Quality gates. A 30-case golden set blocks regressions in CI. The release gate on the real model passed 7 of 8 synthetic cases with 0 unsafe answers; the one miss was over-cautious, choosing escalate instead of re-extract.

7. What we proved live

Every failure type was injected into the real pipeline and handled end to end, between 21 and 27 Oct 2026. Each run went ingest → dbt → gate → engine → fix or person → re-verify. In every run the affected reports were held until their checks passed again.

Case	What the pipeline did	Time
partial_extract (T10)	117 of 10,012 visits missing → re-extract → rebuild → published, no person	1.9 min
source_timeout	First attempt timed out → Airflow retried → 10,012 = receipt, no incident	1.5 min
offsetting_errors	9,970 rows but 9,920 unique → re-extract →	4.3 min

Case	What the pipeline did	Time
	resolved at attempt 1	
late_change	3 claims of 13 Oct reversed after load → 13 Oct rebuilt automatically	6.5 min
reversal_beyond_lookback	Same for 11 Oct, older than the look-back → 11 Oct rebuilt	11.9 min
stale_feed	Replica 30 h behind → WAITING, no data change → caught up → resolved	6.2 min
late_documentation	Only 35% of the day in the source → WAITING → rest arrived → resolved	5.7 min
archived_history	10,012 visits of 12 Oct archived by the EHR → EXPECTED, no alarm, reports published	6.5 min
orphans	2 archived parents → EXPECTED; 3 parents found nowhere → L2; young orphans linked to the unknown member	6.1 min
merged_patient	Three real MPI merges → three Slack cards approved → remap → staging rebuilt → resolved	7.3 min
unit_change (T11)	Glucose in mmol/L → held, L3, never automatic	—
value_mismatch	14 lab values rounded → lab report held, L3; other reports published	6.5 min
deployment_regression	A release doubled 25 visits in a join → release attached, claims report held, 0 data actions	5.0 min
build_error	dbt build failed → all 3 reports held, 0 data actions → resolved after a clean build	2.9 min
staging_duplicates, join_fanout, schema_drift	Escalated; only the claims report held	3.3 min each
source_outage (T19)	BILLING breaker open → one parent incident, 3 children held, EHR reports kept publishing	11.6 min
global_outage	Two sources down → every report held → reload → all resolved	14.0 min

Time = from injection to RESOLVED or ESCALATED, including the pipeline's own build time.

The live runs found 8 bugs that unit tests had missed. For example, the approval worker rebuilt only the latest of several approved dates, and the Slack card never said how the fix ended. All 8 are fixed. That is why every row was proven end to end, not only in tests.

Operations numbers on 27 Oct: 77 incidents in 7 days, 19 handled with no person, 3.9 min median to resolve, all 4 reports published.

8. What people see: console and Slack

Operators use one private web console; data owners only need Slack. The console is FastAPI plus React on the private Cloud Run service `reconcile-console`. You open it through an authenticated proxy (`make console`, then http://localhost:8081). Its API reads data but changes nothing, except replay and retry, which go through the engine.

Screen	What it shows
Operations	The whole pipeline on one page: today at a glance, headline numbers, a 7-day "is today's report trustworthy?" grid, how failures were handled, what the engine may do, recent incidents with replay, and every agent call
Pipeline story (`/story`)	Replays a real incident on a map of the pipeline, step by step, for demos
Incidents, Agents and approvals	The five-gate trace, evidence and action journal; cards waiting, decided or expired
Environment, Sources, Reconciliation, Publication	Cloud resources and IAM, receipts per load, checks per day, and what each report is showing

Every page shows a **red bar while an outage breaker is open** and a **"N cards waiting for you"** badge while an approval is pending. The bar and badge come from `GET /api/operations/shell`, polled every 30 seconds.

Slack, for the data owner (L2)

1. The engine writes an approval card: what failed, the proposed fix and its evidence references. The card shows counts only, never patient identifiers.
2. The owner presses **Approve** or **Reject**. The public callback service `reconcile-slack-callback` checks Slack's signature and records the decision.
3. The approval worker re-checks everything (section 5) and starts one rebuild per approved date.
4. When the incident ends, the card is rewritten with the outcome and a thread reply says **RESOLVED** or **ESCALATED**.

Cards expire if nobody decides; an expired or stale card can no longer start a fix.

How the parts talk. Every flow uses BigQuery tables plus polling: the incident journal, approvals, breakers and agent calls. The Pub/Sub topics from Phase 0 exist but nothing publishes to them yet, so there are fewer moving parts to fail in a POC.

9. Run it

A new environment takes about 30 minutes, mostly waiting for Cloud SQL. You need a Google Cloud project with billing (the free trial works), the gcloud CLI, Terraform 1.6 or later, Docker Desktop with 4 GB or more, Python 3.12 and Node 22. On Windows, run everything in Git Bash, except `git push`, which goes from PowerShell.

First-time setup

```
gcloud auth login && gcloud auth application-default login
gcloud auth application-default set-quota-project $PROJECT
cp infra/terraform.tfvars.example infra/terraform.tfvars   # project_id,
operator_email
make tf-init && make tf-plan && make tf-apply             # Terraform only
from the operator's laptop
make env                                                  # writes .env from
Terraform outputs
make up                                                   # Airflow +
synthetic sources, http://localhost:8080
make generate DATE=2026-10-13 DAYS=3
make deploy-console && make console                       #
http://localhost:8081
scripts/deploy_agent_service.sh                           # private ADK
service (agents)
```

Every day

```
G="docker compose exec airflow bash -c"
make trigger DATE=2026-10-13                  # ingest one day (then pause
the ingest DAGs again)
$G "cd /opt/reconcile/ingest && python -m reconcile_ingest.engine status"   #
incidents at a glance
make replay DATASET=encounters DATE=2026-10-13
```

Prove it

```
scripts/failure_suite.sh list                 # every case, what it does,
what is expected
scripts/failure_suite.sh late_change          # one case (5–15 min)
scripts/failure_suite.sh pkg1                 # outage and freshness group
(about 1 h)
$G "cd /opt/reconcile/ingest && python -m reconcile_ingest.phi_scan"        #
T26: expect 0 findings
scripts/hardening_check.sh                    # laptop + container checks;
exit 1 on any FAIL
```

Demo in 10 minutes. Open Pipeline story and play four incidents. `late_change` fixes itself. `merged_patient` needs a person to approve in Slack. `value_mismatch` is never automatic. `source_outage` becomes one incident for an outage. Then show the Operations numbers.

House rules

- Never commit `.env`, `infra/terraform.tfstate*`, `infra/terraform.tfvars` or helper dump files (`*_files.txt`).
- Keep the ingest DAG toggles **off**, and only one person ingests at a time.

- Never commit `policy.yml` with `kill_switch: true`. Keep `lock_archive_retention = false` (locking cannot be undone).
- Synthetic data only. Never paste credentials or tokens into screenshots or chat.
- Pause Cloud SQL when idle, since it is the only continuous cost (about $10 a month): `gcloud sql instances patch reconcile-state --activation-policy=NEVER`.

10. Troubleshooting

Most problems are permissions that are not ready yet, a Windows path, or a file left out of a build. The full list of about 40 rows is in the detailed runbook (`docs/RUNBOOK.md`, section 19).

Symptom	Fix
`terraform apply` fails on Cloud SQL, service networking or "API has not been used"	Peering or APIs not ready: enable the APIs, wait 2 min, run `make tf-apply` again
`iam.serviceAccounts.getAccessToken` denied	Check `operator_email`, `make tf-apply`, wait 1–2 min
Terraform "Duplicate resource"	Keep one declaration (the budget lives only in `budget.tf`)
Cloud Run build: "COPY failed: file not found in build context"	Add `!<folder>/` and `!<folder>/**` to that folder's `.gcloudignore`
Airflow tasks cannot authenticate (Windows)	Set `GCLOUD_CONFIG_DIR` to `$(cygpath -m "$APPDATA")/gcloud`, then `make down && make up`
`CRYPT_E_REVOCATION_OFFLINE`; `--conf` JSON errors	Use curl `--ssl-no-revoke`; push from PowerShell; run `docker compose` in Git Bash
Ingest run stays queued, or 01:00 IST run loads 0 rows	The DAG toggle is off, or was left on: switch on for the run, off afterwards
Incident still waiting after approval	Check the `approval_worker` and `dbt_build` runs; one rebuild runs per approved date
Agent proposals rejected	Expected when confidence or evidence is low; the reason is in `recovery.agent_calls`

11. Status, open items and the path to production

All seven phases are built; Phase 0 and Phase 6 each have one item left before they are signed off.

Phase	What it built	Exit gate
0 Foundations	Terraform environment, identities, synthetic generator, policy, console shell, CI	3 of 4: rebuild proof open

Phase	What it built	Exit gate
1 Ingest	Receipts in the same snapshot, write-once archive, idempotent loads, replay	Passed 21 Oct
2 dbt checks	Staging and marts in `audit`, reconciliation against receipts, value-level lab checks with KMS	Passed 22 Oct
3 Publication gate	Write-Audit-Publish, dependency-aware holds, backups and rollback	Passed 23 Oct
4 Decision engine	Five gates, catalog actions, attempt cap, kill switch, incident journal	Passed 23 Oct
5 Agents and approvals	Diagnosis and Communication agents, Model Armor, ADK on Cloud Run, Slack approvals	Passed 27 Oct
6 Breaker and hardening	Outage breaker, late changes, orphans, Operations and Pipeline story, hardening pass	23 of 23 proven; demo recording open

Open items

- ☐ Switch on the read-only assistant: redeploy the agent service, apply Terraform, set `enabled: true` in the registry, redeploy the console
- ☐ Phase 0 rebuild proof: destroy and rebuild, then compare data fingerprints before and after. The archive keeps objects for 7 days, so the bucket cannot be deleted before about 3 Nov. The recommended scope is everything except the archive bucket.
- ☐ Record the demo
- ☐ Rotate the Slack bot token by hand
- ☐ Add the manual-process comparison to the KPI baseline

Where the POC differs from the target design, and why

To keep cost and moving parts low on a personal trial account, the POC runs Airflow in Docker instead of Composer. It uses one project, without VPC Service Controls or IAP. The engine runs as Airflow DAGs with its journal in BigQuery instead of Cloud Run and Pub/Sub. Cloud SQL is zonal, and the archive bucket is not locked. None of these changes the decisions; each is a step on the path below.

Path to production

1. Move to an organisation account with separate data, control and agent projects, VPC Service Controls and IAP.
2. Connect one real source read-only with masked data.
3. Get written approval for India residency, BAA cover and no training on prompts; only then set `agents.enabled`.
4. Agree L2 and L3 response targets and a freshness deadline per report with the business (for example, the 06:00 bed meeting).

5. Move Airflow to Composer, turn on Cloud SQL HA, and set up disaster recovery in asia-south2 (Delhi).
6. Replace the illustrative thresholds with ones measured on real volumes, and decide between deterministic encryption and HMAC with security.
7. Connect Splunk for logs, and lock the archive bucket.

12. Glossary

Term	Meaning
Business date	The hospital's India day; sources store UTC, so an early-morning visit is "yesterday" in UTC
Receipt (control totals)	Rows, distinct IDs, empty amounts and the ₹ total, counted in the same snapshot as the extract
Write-Audit-Publish	Build in `audit`, check it, only then copy to `analytics`
Held	A report stays on its last verified day, marked as delayed
Catalog action	One of a short, fixed list of safe fixes; `full_refresh`, `rollback`, `source_delete` are never automatic
L1 / L2 / L3	Fixes itself / the data owner approves / engineers fix code
Kill switch	One setting that turns every automatic fix into a recommendation
Outage breaker	Holds a failing source and groups its failures into one incident
Young / true orphan	A child row whose parent has not arrived yet / exists nowhere
Merged patient	Two patient IDs combined by the MPI; old records are remapped to the surviving ID
Look-back	dbt re-reads the last 3 days on every run to catch late changes
ADK, Model Armor	Google's Agent Development Kit (serves the agents on Cloud Run); Google's screening of prompts and answers
T26	The scan proving no patient data reached agent calls, cards or the journal

Further reading: the detailed runbook (`docs/RUNBOOK.md` and the Word and PDF versions, v2.0) covers every phase, every command and every result.